Representations of Sets and Multisets in Constraint Programming

Christopher Jefferson and Alan M. Frisch

AI Group, Department of Computer Science
University of York, UK
[caj,frisch]@cs.york.ac.uk

Abstract. Constraint programming is a powerful and general purpose tool, but its use is limited, as the process of refining a specification of a problem into an efficient constraint program (known as modelling) is more of an art than a science at present and must be learned by years of experience. This paper theoretically analyses one frequently occurring pattern in modelling, how to choose between different representations of high-level structures, in particular sets and multisets. It differs from previous work by providing methods of comparing very different representations, and by abstracting away from a particular implementation of a representation. It demonstrates useful theoretical dominance results between representations in both a problem dependant and independent context.[1]

1 Introduction

Constraint programming has proved to be an efficient method of solving combinatorial problems, and there are now a number of powerful and efficient constraint solvers available. Unfortunately, constraint programs which implement the same problem in different ways can have hugely differing runtimes and search size.

The first, and arguably most important part of the modelling process is deciding how to represent each variable in a problem specification, either by creating a CSP where some variables of the original specification are represented as multiple CSP variables or choosing how the constraint solver will internally represent each variable. This decision is affected by a number of issues, including the constraints, the domain size of the variables and if the solver being used offers special support for some choices of representation.

Many previous papers have looked at the most efficient methods of implementing one or more representations of high-level structures, including functions [6], permutations [7] and sets and multisets [1]. These papers compared the search size and runtime required for the implementation of constraints on particular representations, making use of the specialised constraints available in specific solvers, and how logically equivalent constraints on the same variables achieve varying levels of propagation. The purpose of this paper is to avoid

[1] Thanks to both Ian Miguel and Bernadette Martinez-Hernandez for helpful discussions on this work

studying a particular implementation of a representation, and instead given the CSP variables which will represent a particular variable in an abstract specification, investigate how well the best implementation which used those variables would perform in terms of search size. The closest previous work to this paper, by Walsh [8], compared two representations of multisets, and this paper extends some of the results in that paper. This paper begins by giving a more general definition of representation than the one considered in [8], and then goes on to prove a number of useful results, including when representations dominate each other, where one representation always produces smaller searches than another, and also how representations can be shown to produce as small a search as the best possible representation (which is the representation which can represent all possible sub-domains of a variable). The aim is that this theory will be useful to both modellers and automated modelling tools (such as CONJURE [3]), to provide guidance on the best representation to use for a particular problem. The examples used throughout this paper involve sets and multisets, as these are very frequently occurring constructs in constraint programming. The theory however applies to representing any type of variable, and has been usefully applied to a large range of constructs including graphs, relations and spare matrices.

1.1 Background

An instance of a finite domain CSP consists of a triple $\langle \boldsymbol{V}, Dom, C\rangle$, where $\boldsymbol{V}$ is a finite set of variables, Dom is a function which maps each variable $v \in \boldsymbol{V}$ to the finite set of assignments of that variable and C is a finite set of constraints. Each constraint $c \in C$ specifies a set of assignments to some subset of $\boldsymbol{V}$, denoted by $vars(C)$, which the constraint allows.

Given a CSP variable X with domain D, a sub-domain of X is defined as a subset of D. For a vector of CSP variables $\boldsymbol{V}$, a sub-domain of $\boldsymbol{V}$ is defined as a vector $\boldsymbol{VD}$, where $VD[i]$ is a sub-domain of $V[i]$. Given a CSP instance $P = \langle \boldsymbol{V}, D, C\rangle$, a sub-domain of P is a sub-domain of $\boldsymbol{V}$. Given a sub-domain of either a variable, vector of variables or a CSP, the assignments allowed by a sub-domain (denoted $assign(S)$) are defined only on the variables in the sub-domain, and consists of all possible assignments to the variables allowed by the given sub-domains. One sub-domain is (strictly) contained in another if the set of assignments allowed by the first is (strictly) contained in the assignments allowed by the second. A solution of a CSP $\langle \boldsymbol{V}, Dom, C\rangle$ is an assignment to $\boldsymbol{V}$ which satisfies all the constraints in C.

Example 1. A finite domain CSP is given by $\langle\{X, Y\}, \{X \in \{1, 2, 3\} Y \in \{1, 2, 3\}\}, \{X + Y > 1, X < 3\}\rangle$. The domain of X is $\{1, 2, 3\}$. One sub-domain of the CSP is $X \in \{1, 2\}, Y \in \{2, 3\}$, which allows the assignments $\langle X, Y\rangle \in \{\langle 1, 2\rangle, \langle 1, 3\rangle, \langle 2, 2\rangle, \langle 2, 3\rangle\}$. Of these assignments, $\langle 1, 2\rangle$ is the only solution, as it is the only one which satisfies both constraints.

The method for solving CSPs considered in the major of this paper will be branch-and-propagate search. As can be implied from the title, there are two major operations which must be performed while using this kind of search, branching and propagating. Both act on sub-domains of the CSP.

Definition 1. *A* propagation *algorithm is defined by a function F which maps between sub-domains of a CSP with satisfies the following two conditions.*

- $f(S) = T \rightarrow assign(T) \subseteq assign(S)$ *and any solutions in* $assign(S)$ *are also in* $assign(T)$
- $assign(S) \subseteq assign(T) \rightarrow assign(f(S)) \subseteq assign(f(T))$

One common family of propagation algorithms are constraint-based propagation algorithms, which given a sub-domain of a CSP P and a constraint $c \in P$, only remove values which do not satisfy c, and therefore clearly can't be part of any assignment which satisfies P. *MAC propagation* [2] removes all such values, and when no such values can be removed the sub-domain is defined to satisfy GAC with respect to c.

The second operation used during search is branching, which simply takes a sub-domain of a CSP P and generates n new CSPs $P_1, \ldots, P_n$ where P_i is identical to P except with an added constraint C_i, such that all assignments in the sub-domain are allowed by at least one of the C_i. Commonly the C_i are unary constraints. If any constraints can be used for branching it is possible for search to not be finite. The simplest way to prevent this is to ensure either at each propagation at least one value is removed from the sub-domain of some variable in each branch, or alternatively one finite CSPs, to ensure that no constraint is imposed more than once.

Definition 2. N-way branch-and-propagate search *of a CSP P begins from the complete sub-domain of P and is defined recursively. Given a sub-domain of a CSP, branch-and-propagate search first performs propagation algorithms on the sub-domain. If the sub-domain of each variable consists of exactly one value, then this is an* assignment node*. If this assignment satisfies all the constraints then a solution has been found, else this is a* failure node. Failure nodes *also arise if the sub-domain of any variable becomes empty. If neither of these conditions hold then the current sub-domain is* branched*, generating a set of new CSPs to solve.*

2 Representations

Given a specification of a problem, it is frequently not possible to map it directly into a CSP as it contains variables of types which are not supported by the constraint solver being used.

Example 2. The Social Golfer Problem involves trying to schedule w rounds of golf, where in each round the $g \times s$ players are arranged into g groups of size s such that no pair of players appear in a group together more than once.

The most obvious formulation of Example 2 would consist of a single variable to represent the schedule. This leads to variables with huge domain, for example 16 golfers split into 4 games on each of 6 weeks would lead to a schedule variable with over $6 * 10^{45}$ possible assignments. For this, and a number of other reasons, it often not efficient or even feasible to use an initial choice of representation. There are a number of ways of reducing memory usage to feasible amount, each of which has it's own trade-offs. Studying these trade-offs is the major purpose of this paper.

Example 3. Consider a CSP variable X with domain $D = \{1, 2, \ldots, 10\}$. The "bounds representation" of X allows only the sub-domains of X (and therefore subsets of D) represented by the set $\{\langle l, u\rangle | 1 \leq l \leq u \leq n\} \cup \{\emptyset\}$, where $\langle l, u\rangle$ represents the sub-domain $\{x | l \leq x \leq u\}$ and $\emptyset$ represents the empty sub-domain.

Propagating the constraint $X > 5$ from the complete sub-domain of X as much as possible leads to the sub-domain $\{6, 7, 8, 9, 10\}$, which the "bounds representation" represents exactly with $\langle 6, 10\rangle$. Consider now the constraint "X is even". The assignments to X allowed by this constraint are $\{2, 4, 6, 8, 10\}$. Any element of the "bounds representation" which allows all these assignments (for example $\langle 2, 10\rangle$) would also allow the set of assignments $\{3, 5, 7, 9\}$.

Example 4. Consider a CSP variable X with the domain "all subsets of {0,1,2,3} of size 2". This is represented under the "explicit representation" by 2 variables, V_1 and V_2, each of domain $\{0, 1, 2, 3\}$, and the constraint $V_1 \neq V_2$. Assignments to V_1 and V_2 represent assignments to X by the mapping $X = \{V_1, V_2\}$.

Consider the problem "find all assignments to X such that $1 \notin X$". This would be represented with the explicit representation by the CSP "find all assignments to V_1 and V_2 such that $V_1 \neq 1, V_2 \neq 1$ and $V_1 \neq V_2$". At the first node of search, MAC would remove 1 from the sub-domain of V_1 and V_2, and the remaining assignments to V_1 and V_2 which satisfy $V_1 \neq V_2$ all represent assignments to X which satisfy $1 \notin X$.

Consider now the problem "find all assignments to X such that the sum of the members of X is 3" This would be represented by the CSP "find all assignments to V_1 and V_2 such that $V_1 \neq V_2$ and $V_1 + V_2 = 3$". At the first node of search, no values can be removed from the domains of either V_1 or V_2, as they are all part of an assignment to the variables which satisfies all the constraints. There are however many assignments which represent assignments to X whose sum is not 3, for example $V_1 = 2, V_2 = 3$.

Example 5. Given a natural number n, consider the CSP containing a single set variable A with domain $MS(\{1, \ldots, 2n\}, 1, [0 \ldots 2n])$ and the two constraints $|A| = n$ and $|A| = n + 1$. If a solver can achieve GAC propagation for these two constraints, propagating both constraints will empty the sub-domain of A without search. Now consider A being represented by a Boolean vector $\boldsymbol{V}$ of length $2n$, where V[i] is true is $i \in A$. On any sub-domain of V where less than $n - 1$ variables are instantiated, neither the constraint $sum(\boldsymbol{V}) = n$ or $sum(\boldsymbol{V}) = n + 1$ would remove any values from the domains of any variables if propagated, so the size of the search tree will be at least 2^{n-2} nodes.

Examples 3 and 4 give two examples of representing a variable in a way which limits the number of possible sub-domains which are allowed. These aim to show the problems involved with choosing how to represent a variable in a CSP. In both examples, the first constraint considered is unaffected by the choice of representation, as it is possible to represent the sub-domain of the original variable which contains only those assignments which satisfy the constraint. In both examples, the second constraint considered suffers by the choice of representation, as the smallest sub-domain which contains all assignments which satisfy the constraint also contains a number of assignments which do not. As the sub-domain of the CSP is the only method by which propagation algorithms can pass information to each other, this can impair the performance of search, and lead to greatly increased search size. This is shown in Example 5, where a poor choice of representation of a set of size n causes search to increase in size from a single node to a search tree of size exponential in n. This paper aims to formalise the idea of a representation and investigate how the states which can be represented limit the effectiveness of a particular representation.

Example 3 shows an example of where the representation can be considered as integral to the solver. Example 4 on the other hand, generates a new CSP whose solutions can be mapped to the solutions of the original. These should not be considered as disjoint families of representations, as the second can be considered as a subset of the first, and many solvers implement nested types internally as directly as a vector of CSP variables. For example, Conjunto[4] represents sets using a representation known as the occurrence representation (Definition 8). Both of these kinds of representations will be considered, and much of the theory of representations which will be presented applies equally to all representations, although some results can be extended or simplified by considering only representations which map one CSP to another.

Definition 3. *A representation of a CSP domain D is defined as a pair $\langle R, f \rangle$, where R is a partially ordered set and $f : R \to \mathbb{P}(D)$. R is the set of states which the representation can take, and for each $r \in R$, $f(r)$ is the sub-domain of X this state represents. A state $r_1 \in R$ is defined to be reachable from a state $r_2 \in R$ if $r_2 < r_1$. To be a valid representation, $\langle R, f \rangle$ must satisfy the following conditions:*

1. $r_1 \leq r_2 \to f(r_1) \subseteq f(r_2)$
   If r_1 is reachable from r_2, then r_1 must represent a subset of the assignments allowed by r_2.
2. $\exists 1\ r_D \in R.\ (f(r_D) = D \wedge \forall r \in R.\ r \neq r_D \to r < r_D)$
   There must be some initial representational state from which all other states can be reached, to begin search.
3. $\forall r \in R.\ \exists r_\emptyset \in R.\ (f(r_\emptyset) = \emptyset \wedge r_\emptyset \leq r)$
   From all representational states it must be possible to reach the state which represents nothing, else it would not be possible to fail.
4. $\forall r \in R, \forall d \in f(r).\ \exists r_d \in R.\ f(r_d) = \{d\} \wedge r_d \leq r.$
   If a representational state represents an assignment d, it must be possible to reach a state which represents just the assignment d, else it would not be possible to reach all assignments a representation state represents.

Definition 4. *A simple representation of a CSP domain D is a representation $\langle R, f \rangle$ where the extra condition $\forall r_1, r_2 \in R.\ f(r_1) \subseteq f(r_2) \to r_1 \leq r_2$ holds. Combining this with Condition 1 of the definition of representation gives $\forall r_1, r_2 \in R.\ f(r_1) \subseteq f(r_2) \leftrightarrow r_1 \leq r_2$.*

The definition of *n-way branch-and-propagate search* in Section 1.1 did not refer to representations. The addition of representations limits the search, so that only certain sub-domains are allowed during search, and also which sub-domains may be reached from a given sub-domains is limited by the representation. The most important change to the definition of propagation algorithms is that instead of the result of applying the propagation algorithm being a subset of the assignments, instead the state of search achieved must be less in the ordering on the representation than the state which before. A proof ensuring that the limitations places on representations in Definition 3 are sufficient to ensure search is still correct is outlined in Theorem 1 due to lack of space. We are now in a position to justify the definition of representation given in Definition 3, and also prove that this definition is sufficient for search to be correct.

Theorem 1. *The four conditions in Definition 3 are necessary and sufficient for a representation to be used to represent sub-domains in an n-way branch and propagate search and any branching strategy still leads to all solutions, finite search, and all nodes being either solutions nodes, failure nodes, or branched on.*

Proof. If all these conditions are satisfied, then by condition 2, the first node can allow all assignments to the CSP. Condition 1 ensures search will be finite, condition 3 ensures from any node failure can be reached and condition 4 ensures from any node it possible (although not necessary) to choose a variable whose sub-domain allows more than one assignment and generate a new node for every allowed assignment. □

2.1 Variable representations

As discussed by example at the beginning of Section 2, an important family of representations are those which represent a CSP variable by replacing it with a vector of CSP variables and mapping constraints involving the original variable to new constraints involving the new variables. These are known as *variable representations* (Definition 5). One important feature of variable representations is that as the result of applying them is another CSP and the resulting variables can be replaced with representations themselves.

Definition 5. *A* variable representation *of a variable X with domain D is a pair $\langle \boldsymbol{V}, f\rangle$ where $\boldsymbol{V}$ is a vector of CSP variables and f is a partial surjective function from assignments of $\boldsymbol{V}$ to D. An assignment to $\boldsymbol{V}$ which is in the domain of f is said to* represent *its image under f. The representation constraint is defined as the constraint "$\boldsymbol{V}$ is in the domain of f". For a sub-domain $\boldsymbol{V'}$ of $\boldsymbol{V}$, $f(\boldsymbol{V'})$ is defined as the sub-domain of X generated by applying f to all assignments in $\boldsymbol{V'}$.*

The induced representation of a variable representation $\langle \boldsymbol{V}, f\rangle$ *is defined to be the representation $\langle R, g\rangle$ where R is defined as all sub-domains of $\boldsymbol{V}$, R is ordered by $r_1 \leq r_2 \leftrightarrow \forall v \in V.\ r_1[v] \subseteq r_2[v]$ and* $g(r) = \{f(x) | x \text{ is allowed by } r\}$.

Lemma 1. *The representation $\langle R, g\rangle$ induced from a variable representation $\langle \boldsymbol{V}, f\rangle$ is simple if and only if f is injective.*

Proof. If the function f is not injective, then there must exist two distinct assignments v and w of $\boldsymbol{V}$ such that $f(v) = f(w)$.The sub-domains containing only these assignments will clearly be incomparable but represent the same set of assignments. Consider now the case where f is injective. If $v \leq w$ then w allows all assignments v does, and therefore $f(v) \subset f(w)$. Similarly if $f(v) \subseteq f(w)$ then as f is injective $v \leq w$. □

Definition 6. *Given a constraint C and a variable representation $\langle \boldsymbol{V}, f\rangle$ of a variable $X \in vars(C)$, then X is defined to be replaced in C by $\langle \boldsymbol{V}, f\rangle$ as follows:*

Given $vars(C) = \{X, Y_1, \ldots, Y_n\}$, then C can be expressed as the subset S of $\langle D(X) \times D(Y_1) \times \cdots \times D(Y_n)\rangle$ which contains the assignments $vars(C)$ which are allowed by C. C is the replaced by a new constraint over the V_i and Y_i which allows the assignments $\{\langle v_1, \ldots, v_m, y_1, \ldots, y_n\rangle |\ \langle f(\langle v_1, \ldots, v_m\rangle), y_1, \ldots, y_n\rangle \in S\}$. □

Example 6. Consider a set variable X of size 2 drawn from $\{0, 1, 2, 3\}$. This is *represented* under the *explicit* representation by $\langle \boldsymbol{V}, f\rangle$, where $\boldsymbol{V} = \langle V_1, V_2\rangle$ and the V_i have domain $\{0, 1, 2, 3\}$, f is defined as mapping each pair of values $\langle V_1, V_2\rangle$ to the set $\{V_1, V_2\}$ where X and Y are distinct. Consider the constraint $\sum_{i \in X} = 3$, expressed as a set of tuples with $\{\langle\{0, 3\}\rangle, \langle\{1, 2\}\rangle\}$. When this constraint is replaced by the representation $\langle \boldsymbol{V}, f\rangle$, then the original constraint is replaced by the constraint $\langle V[1], V[2]\rangle \in \{\langle 0, 3\rangle, \langle 3, 0\rangle, \langle 1, 2\rangle, \langle 2, 1\rangle\}$.

Theorem 2. *a) Given a CSP P, a variable X from P and a variable representation $\langle \boldsymbol{V}, f\rangle$ of X, a new CSP P' can be generated by adding the variables $\boldsymbol{V}$ and the constraint $f(\boldsymbol{V}) = X$ to P. All solutions of P' can be generated by taking a solution of P and then adding an assignment to $\boldsymbol{V}$ which satisfies the constraint $f(\boldsymbol{V}) = X$, and there will be at least one such assignment to $\boldsymbol{V}$ for each assignment to X, and exactly one when the representation is simple.*

b) Given a CSP P which contains a variable X, a vector of variables $\boldsymbol{V}$ and the constraint $c_R = f(\boldsymbol{V}) = X$ where $\langle \boldsymbol{V}, f\rangle$ is a representation of X, a new CSP P' can be generated by taking any constraint C in P (except c_R) where $X \in vars(C)$ and replacing X with $\boldsymbol{V}$ by $\langle \boldsymbol{V}, f\rangle$ in C. The solutions of P and P' will be are identical.

c) Given a CSP P which contains a vector of variables $\boldsymbol{V}$ and a variable X, which is in only the constraint $f(\boldsymbol{V}) = X$ where $\langle \boldsymbol{V}, f\rangle$, a new CSP P' can be generated by removing the variable X and the constraint $f(\boldsymbol{V}) = X$. The solutions to P' are exactly the solutions to P without the assignment to X.

Proof. a) Trivial, as for each assignment to X there exists at least one (and exactly one if the representation is perfect) assignment to $\boldsymbol{V}$ such that $f(\boldsymbol{V}) = X$ is satisfied.

b) Consider an assignment s to $vars(P)$ which is a solution to P. This will be a solution to P' if and only if s satisfies C with X replaced by $\boldsymbol{V}$, as described in Definition 6. However we know that $vars(P)$ satisfies C, and therefore in particular this assignment satisfies $f(\boldsymbol{V}) = X$. By the definition of replacing a variable in a constraint with a representation, the original constraint will be satisfied if and only if the new one is. The reverse argument follows identically.

c) Clearly any solution to P when limited to $vars(P')$ will be a solution to P', as the constraints on P' are a subset of the constraints on P. Given any solution to P', adding the assignment to X which satisfies $f(\boldsymbol{V}) = X$ which also clearly be a solution to P, as it is only P' with one extra variable and constraint, and this new constraint is the only one which involves X. $\square$

Theorem 2 proves that the Definition 6 can be used to apply representations to a CSP and get a new CSP, whose solutions can be mapped to the solutions of the original CSP. One minor problem with Definition 6 is that it assumes constraints are expressed explicitly. Most constraint solvers operate most efficiently on constraint expressed implicitly using a small language of constraints. How an implicit representation of a constraint can be refined to an implicit representation on a representation is a complex issue in its own right, see [3]. In this paper refined constraints will be given implicitly where possible, but how these implicit constraints could be generated will not be discussed.

2.2 Representations of Sets and Multisets

One of the most common types for variables which occur in specifications of combinatorial problems is sets and multisets, and these will be used as the primary example throughout this paper. There are a number of representations of sets and multisets in common usage in constraint programming, and some of these will be explored in this paper. Definition 7 gives a formal way of referring to the various families of sets and multisets which will arise.

Definition 7. *Given set X, natural number occ and integer range Size, then $S = MS(X, occ, Size)$ denotes the set of all (multi)sets drawn from X with size in the range Size and where each value occurs at most occ times. Three common special cases of this definition are where occ is 1 (so S represents only sets), Size allows a single value (so S represents a single fixed size of (multi)sets), and Size is a range at least as large as $[0, |X| * occ]$, so it places no limit on the (multi)sets in S, so S is unsized.*

Definition 8. *Given a (multi)set variable X with domain $MS(S, occ, Size)$, the occurrence representation of X is defined to be $\langle \boldsymbol{V}, f\rangle$, where $\boldsymbol{V}$ is a vector indexed by S of variables with domain $\{0, \ldots, occ\}$. The function f maps assignments of $\boldsymbol{V}$ to X by $f(\boldsymbol{v}) = \{v[s] \times s | s \in S\}_m$, under the condition the resulting (multi)set is in the domain of X. The representation constraint is therefore given by $sum(V) \in Size$. $\boldsymbol{V}$ is called the occurrence vector.*

Definition 9. *Given a (multi)set variable X with domain $MS(S, occ, Size)$, the explicit representation of X is defined to be $\langle \boldsymbol{E}, f\rangle$, where $\boldsymbol{E}$ is a vector of variables of domain $S \cup \{\emptyset\}$ (where $\emptyset$ represents a value not in S) indexed by the range $1, \ldots, max(size)$. f maps assignments of $\boldsymbol{E}$ by $f(\boldsymbol{v}) = \{E[i] | 1 \leq i \leq max(size) \wedge E[i] \neq \emptyset\}_m$, where this (multi)set is in $MS(S, occ, Size)$. $\boldsymbol{E}$ is known as the* element *vector.*

Definition 10. *Given an abstract variable X with domain $MS(S, occ, Size)$, it is* represented *under the* explicit with check *representation by $\langle \boldsymbol{E}.\boldsymbol{C}, f\rangle$, where*

$\boldsymbol{E}$ and $\boldsymbol{C}$ are both of length $max(size)$, the elements of $\boldsymbol{C}$ are Boolean variables and the elements of $\boldsymbol{E}$ have domain S. The function f maps assignments of $\boldsymbol{E}$ and $\boldsymbol{C}$ to the (multi)set which contains each $E[i]$ for which $C[i]$ is TRUE*, where this (multi)set is in $MS(S, occ, Size)$. $\boldsymbol{E}$ is called the* element *vector, and $\boldsymbol{C}$ is called the* check *vector.*

Lemma 2. *For (multi)sets of domain and size greater than 1 the explicit representation is not simple.*

Proof. It is possible to freely permute the vector of variables in the explicit representation and they will represent the same (multi)set, so as long as this vector has length greater than 1 and more than one possible assignment, the representation cannot be simple. □

The *explicit with check* can be simpler to implement than the *explicit* representation, as it does not require constraints which can cope with a special domain element which represents that variable does not represent a value in the (multi)set. Both these representation have the problem of variable symmetry, as permuting an assignment to the element vector (and identically permuting the check vector in the case of the *explicit with check* representation) generates an assignment which represents the same (multi)set. Only one method of breaking this symmetry shall be considered in this paper, which is to impose that an assignment to the *element* vector only represents a (multi)set when it is ordered according to some ordering of the elements the (multi)set is drawn from. After performing symmetry breaking, both these representations are simple.

Definition 11. *The* Gent representation[2] *was designed to try to combine the strengths of the* occurrence *and* explicit with symmetry breaking *representations. Given a (multi)set variables X with domain $MS(S, occ, size)$, it is represented with the Gent representation by $\langle \boldsymbol{V}, f\rangle$, where $\boldsymbol{V}$ is a vector of length $|S|$ with variables of domain $\{0, 1, \ldots, max(size)\}$. f maps assignments of $\boldsymbol{V}$ to assignments of X by the following rules, assuming a total ordering on the elements of S. 1) If a is the smallest element in S, $f(\boldsymbol{v})$ contains $v[a]$ occurrences of a. 2) If $v[b] \neq 0$, then $f(v)$ contains $v[b] - max\{v[i] | i < b\}$ occurrences of b. 3) $f(v)$ is only defined if $a < b \rightarrow (v[b] = 0 \cup v[a] < v[b])$ and the resulting (multi)set is valid assignment to X.*

Example 7. If $X = MS(\{1, 2, 3, 4, 5\}, 5, [0, 5])$, then if X is represented under the Gent representation then $\langle 0, 2, 0, 4, 0\rangle$ represents $\{2, 2, 4, 4\}_M$ (where $\{\}_M$ denotes a multiset) and $\langle 1, 2, 4, 0, 0\rangle$ represents $\{1, 2, 3, 3\}_M$. The representation of both $\langle 0, 1, 0, 1, 0\rangle$ and $\langle 0, 2, 0, 1, 0\rangle$ is undefined, as the non-zero elements are not in strictly increasing order.

There are a number of variants of these representations, some of which modify them in a minor manner, and others which perform large additions. One very common extension, and the only discussed in this paper, is adding a variable

[2] Discovered by Ian Gent, unpublished

whose value is size of the (multi)set. It is simple to add this variable to each of the representations considered so far. These extended representations will be denoted by adding "+ size" to their title, for example the "occurrence + size representation".

3 Comparing representations

The definition of representations given in Section 2 provides a method of specifying representations and in the case of variable representations using them to map one CSP to another. In order to become useful to modellers, these definitions must be used to aid choosing the "best representation" to use to solve a particular CSP.

The first obvious problem is to decide on a definition of "best representation". In particular, solver-specific implementation issues and optimisations mean that even on different versions of the same solver the representation which will solve a problem fastest can change drastically. The next most obvious method of comparing representations, and the one considered here, is to compare the size of search, ignoring how long a particular implementation may take to implement a given implementation. Of course it is not possible to entirely ignore time and memory usage, in particular because except in extreme cases, using a representation should always decrease memory usage and time taken per node, but may increase (and should never decrease) search size.

The strongest possible relationship between two representations A and B of the same variable would be if it could be shown that using A always produced smaller searches than B. Definition 12 defines exactly this property. It is obvious that given this definition, if A dominates B, then given any search which was performed where a variable is represented using B, B could be replaced with A.

Definition 12. *$Rep_1 = \langle R_1, f_1\rangle$* dominates *$Rep_2 = \langle R_2, f_2\rangle$ (or Rep_2 is* embedded *in Rep_1) if there is a function $M : R_2 \to R_1$ such that $\forall r \in R_2.\ f_1(M(r)) = f_2(r)$ and $\forall r_1, r_2 \in R_2.\ r_1 \leq r_2 \to M(r_1) \leq M(r_2)$. Rep_1 and Rep_2 are* equivalent *if Rep_1 dominates Rep_2 and Rep_2 dominates R_1. Rep_1* strictly dominates *Rep_2 if Rep_1* dominates *Rep_2 and Rep_2 does not* dominate *Rep_1.*

Dominance is very similar to the idea of expressivity from [8], and for simple representations Lemma 3 shows that they are identical. For non-simple representations however is is necessary to also consider the ordering relation between states, as well as the sub-domains those states represent. Considering only the states which can be represented can suggest a representation is more useful than it actually is during search, as Example 8 demonstrates.

Lemma 3. *Given two* simple representations *$Rep_1 = \langle R_1, f_1\rangle$ and $Rep_2 = \langle R_2, f_2\rangle$ then Rep_1 dominates Rep_2 if and only if $\{f_2(r_2) | r_2 \in R_2\} \subseteq \{f_1(r_1) | r_1 \in R_1\}$.*

Proof. If Rep_1 dominates Rep_2, then there exists a function $M : R_2 \to R_1$ such that $\forall r \in R_2.\ f_1(M(r)) = f_2(r)$ and therefore $\forall r_2 \in R_2.\ \exists r_1 \in R_1.\ f_1(r_1) = f_2(r_2)$.

In the opposite direction, as the set of sub-domains represented Rep_2 is a subset of those represented by Rep_1 and as both representations are simple (and so each sub-domain is represented at most once), then it is simple to define a function M from R_2 to R_1 by $f(r_2) = r_1 \leftrightarrow f_1(r_1) = f_2(r_2)$. As the ordering on simple representations is entirely determined by the domains they represent, this function defines the dominance between Rep_1 and Rep_2. $\square$

Example 8. Consider representing two element multisets drawn from $\{1, 2, 3\}$ using the explicit representation with and without symmetry breaking.

The sub-domain $\{\{1, 1\}_M, \{1, 2\}_M, \{2, 3\}_M\}$ can be represented by the sub-domains $\langle\{1, 2\}, \{1, 3\}\rangle$ if symmetry breaking is not imposed, but if symmetry breaking is imposed the smallest sub-domains which contain these assignments are $\langle\{1, 2\}, \{1, 2, 3\}\rangle$, which also allows $\{2, 2\}_M$.

Consider now representing the sub-domain $\{\{1, 2\}_M, \{2, 3\}_M\}$. These can be represented exactly by the sub-domains $\langle\{2\}, \{1, 3\}\rangle$ without symmetry breaking, but require at least the sub-domains $\langle\{1, 2\}, \{2, 3\}\rangle$ with symmetry breaking, which also allow the multiset $\{2, 2\}_M$.

Without symmetry breaking, there are more possible sets of sub-domains which can be represented. This comes at the cost however that the domains which were shown second are not reachable from those which were generated first. It is therefore misleading to simply list the states which are reachable during search in the case of non-simple representations.

3.1 Dominance in variable representations

While dominance is a useful property, proving if one representation dominates another is non-trivial directly from the definition, and therefore finding simpler methods of proving dominance is useful. For simple variable representations, Theorem 3 gives a sufficient, although not necessary, to prove one simple representation dominates another.

Definition 13. *A set of* channelling constraints *between two* variable representations $R_1 = \langle \boldsymbol{V_1}, f_1\rangle$ *and* $R_2 = \langle \boldsymbol{V_2}, f_2\rangle$ *of a variable* X *is a set of constraints over* $\boldsymbol{V_1}$ *and* $\boldsymbol{V_2}$ *such that an instantiation* $\boldsymbol{v_1}$ *of* $\boldsymbol{V_1}$ *and* $\boldsymbol{v_2}$ *of* $\boldsymbol{V_2}$ *satisfies all the constraints if and only if either* $f_1(\boldsymbol{v_1})$ *is equal to* $f_2(\boldsymbol{v_2})$*, or both are undefined.*

Theorem 3. *Given two simple* variable representations $R_1 = \langle \boldsymbol{V}, f\rangle$ *and* $R_2 = \langle \boldsymbol{W}, g\rangle$ *of a variable* X *and a set* C *of channelling constraints between* $\boldsymbol{V}$ *and* $\boldsymbol{W}$*, where each* $c \in C$ *can be expressed in the form* $v = x \leftrightarrow (w_1 = y_1 \vee \cdots \vee w_n = y_n)$ *for constants* $x, y_1, \ldots, y_n$ *and variables* $v \in \boldsymbol{V}$ *and* $w_i \in \boldsymbol{W}$*, then* R_2 *dominates* R_1*.*

Proof. Consider a sub-domain $\boldsymbol{V'}$ of $\boldsymbol{V}$ which is GAC with respect to the representation constraint, and construct the maximal sub-domain $\boldsymbol{W'}$ of $\boldsymbol{W}$ which is GAC with respect to the representation constraint and $\boldsymbol{W'}$ and $\boldsymbol{V'}$ together are GAC with respect to C. By construction, for each assignment $v' \in V'$ where $f(v')$ is defined, there will exist a $w' \in W'$ such that $g(w') = f(v')$.

Consider an assignment $w' \in \boldsymbol{W'}$ such that $g(w')$ is defined. There must exist a unique assignment $v \in \boldsymbol{V}$ such that $g(w') = f(v)$ and $c(w', v)$ holds for all $c \in C$. It remains to show this assignment is allowed by $\boldsymbol{V'}$.

Consider a single channelling constraint in C. if the constraint is true, either both the left and right hand side of the constraint are false, or the comparison on the left hand side and at least one comparison on the right hand side are true. Given a sub-domain of $\boldsymbol{W}$ therefore, by looking at each channelling constraint in turn it is possible to construct a list of assignments to variables in $\boldsymbol{V}$ which must be allowed, and which must be forbidden. Considering a larger sub-domain of $\boldsymbol{W}$ can only increase the number of assignments to variables in $\boldsymbol{V}$ which must be allowed. Therefore as $C(\boldsymbol{v}, \boldsymbol{w})$ is true, then as $\boldsymbol{W'}$ contains $\boldsymbol{w'}$, $\boldsymbol{V'}$ must contain $\boldsymbol{v'}$, because GAC $C(\boldsymbol{V'}, \boldsymbol{W'})$ holds. □

Theorem 4. 1. *The* Gent *representation dominates the occurrence representation for representing sets.*
2. *The* Gent + size *representation dominates the* occurrence + size *representation for representing sets.*
3. *The* Gent *representation dominates the explicit with symmetry breaking representation for fixed sized sets and multisets.*
4. *The* Gent + size *representation dominates the* explicit with symmetry breaking *representation for variable sized sets and multisets.*
5. *The* explicit *and* explicit with check *representations with symmetry breaking are equivalent on variable and fixed sized sets and multisets.*

Proof. From Theorem 3, we only need to find a set of channelling constraints of the appropriate form to prove dominance. The required channelling constraints are given below. In each case, a CSP variable $X = MS(S, occ, size)$ is represented in multiple ways.

1. Represent X as $\langle \boldsymbol{V}, f \rangle$ under the *ordered occurrence* representation and $\langle \boldsymbol{W}, g \rangle$ under the *occurrence* representation. Consider the set of constraints $\{W[i] = 0 \leftrightarrow V[i] = 0 | i \in X\} \cup \{W[i] = 1 \leftrightarrow V[i] = 1 \vee \cdots \vee V[i] = size | i \in X\}$.
2. Represent X as $\langle \boldsymbol{V}.S_1, f \rangle$ under the *ordered occurrence + size* representation (where S_1 is a variable which represents the size of X), and $\langle \boldsymbol{W}.S_2, g \rangle$ under the *occurrence + size* representation. Consider the sets of constraints $\{W[i] = 0 \leftrightarrow V[i] = 0 | i \in X\} \cup \{W[i] = 1 \leftrightarrow V[i] = 1 \vee \cdots \vee V[i] = size | i \in X\}$ and $\{S_1 = i \leftrightarrow S_2 = i | i \in size\}$.
3. Represent X as $\langle \boldsymbol{V}, f \rangle$ under the *ordered occurrence* representation and $\langle \boldsymbol{W}, g \rangle$ under the *explicit* representation. Consider the set of constraints $\{W[i] = x \leftrightarrow V[x] = \{i | x \in X, i \in \{1, \ldots, size\}\} \cup \{W[i] = \emptyset \leftrightarrow \textsc{False} | i \in \{1, \ldots, size\}$.
4. Represent X as $\langle \boldsymbol{V}.S, f \rangle$ under the *ordered occurrence + size* representation and as $\langle \boldsymbol{W}, f \rangle$ under the *explicit* representation. Consider the set of constraints $\{W[i] = x \leftrightarrow V[x] = i | x \in X\} \cup \{W[i] = \emptyset \leftrightarrow S = 0 \vee \cdots \vee C = i | i \in size\}$.
5. Represent X as $\langle \boldsymbol{V}, f \rangle$ under the *explicit* representation and as $\langle \boldsymbol{W}.\boldsymbol{C}, g \rangle$ under the *explicit with check* representation. The set of constraints $\{V[i] =$

$x \leftrightarrow W[i] = x | x \in X, i \in size\} \cup \{V[i] = \emptyset \leftrightarrow C[i] = 1 | x \in X$ show that the *explicit with check* representation dominates the *explicit* representation and the set of constraints $\{C[i] = 0 \leftrightarrow V[i] = \emptyset | i \in size\} \cup \{C[i] = 1 \leftrightarrow V[i] \in X | i \in size\} \cup \{W[i] = x \leftrightarrow V[i] = x \vee V[i] = \emptyset | i \in size, x \in X\}$ demonstrates the converse.

□

4 Perfect representations

While dominance provides a powerful method of comparing representations, it is often too coarse. A more precise system would compare representations with respect to a specific problem. Unfortunately, performing this usefully in general has proved difficult, but this section studies a useful special case of this problem, showing when a representation of some variable is equivalent to the complete representation with respect to one or more constraints.

Definition 14. *Given a constraint C, a set $T = \{\langle R_v, f_v \rangle\}$ of representations for some subset of $vars(C)$, some $\langle R_t, f_t \rangle \in T$ which represents a variable X, and the constraint C' obtained by applying each representation in T to variables in $vars(C)$, then "$\langle R_t, f_t \rangle$ is perfect with respect to C and T" if given a sub-domain A of $vars(C')$ which is GAC with respect to C', then any assignment to R_t which is allowed by A and has an image under f_t can be extended to an assignment to A which satisfies C'.*

Note that the definition of perfect considers a set of representations. This is because replacing many variables with representations may be perfect with respect to each of these representations, while replacing any single variable with a representation is not.

It follows from the definition of perfect that using a perfect representation instead of the original variable will not lead to an increase in search space assuming GAC propagation as whenever GAC propagation occurs, all assignments which represent an assignment to the original variable and which do not satisfy the constraint will be removed.

Example 9. Consider a variable A with domain $MS(\{1, \ldots, n\}, 1, [0 \ldots n))$ represented with the *occurrence* representation with *occurrence* vector $\boldsymbol{V}$. The constraint $a \in A$ for a constant value a is mapped via the representation to the constraint $V[a] = 1$. Any sub-domain of $\boldsymbol{V}$ which is GAC with respect to this constraint will clearly have $V[a]$ with only 1 in its domain and therefore any assignment to $\boldsymbol{V}$ which has represents an element of A will satisfy the original constraint. The *occurrence* representation is therefore perfect with respect to the constraint $a \in A$.

Example 9 gives an example of a perfect representation with respect to a specific constraint, while Example 5 gave an example of a non-perfect one, which leads to an exponential increase in search size compared to using the complete representation. In a similar fashion to Theorem 3, which showed a useful sufficient condition for one variable representation to dominate another, Theorem 5 shows a necessary and sufficient (although only sufficient is proved due to space restrictions) condition for a variable representation to be perfect with respect to a given constraint.

Definition 15. *Given a constraint C and a set of constraints K, then a set of constraints S is defined to be a* split *of C in the context of K if $vars(C) = \cup\{vars(s)|s \in S\}$, $(s,t \in S \wedge s \neq t) \rightarrow vars(s) \cap vars(t) = \emptyset$ and $C \wedge K$ is logically equivalent to $\wedge\{s|s \in S\} \wedge K$.*

Lemma 4. *Given a split S of a constraint C in the context of K, then the set of constraints S' generated by creating a new constraint s' for each each $s \in S$ where $vars(s')$ is the same as $vars(s)$ and an assignment is allowed by s' if it can be extended to a valid assignment of C is also a split of C in the context of K.*

Proof. There cannot be an assignment to $vars(C)$ which is allowed by $C \wedge K$ and not by $(\wedge S') \wedge K$ as each $s' \in S'$ accepts any assignment which can be extended to a complete assignment of $vars(C)$ which satisfies C. Furthermore, all assignments to $(\wedge S') \wedge K$ must be allowed by $(\wedge S) \wedge K$ as the constraints in S' allow the minimal possible set of assignments required to satisfy that they are a split. Therefore $C \wedge K \rightarrow (\wedge S') \wedge K \rightarrow (\wedge S) \wedge K \leftrightarrow C \wedge K$, and therefore all 3 must be equivalent. □

Theorem 5. *Given a constraint C and the constraint C' generated by applying a set of variable representations $R = \{\boldsymbol{V_i}, f_i\}$ to C, a distinguished representation $\langle \boldsymbol{V_r}, f_r \rangle \in R$, and the set of constraints K generated by taking the representation constraints of each element of R, then r is* perfect *with respect to C and R if C' can be* split *into a set of constraints S in the context of K, each of which contains at most one variable from each $\boldsymbol{V_r}$.*

Proof. Without loss of generality, we will assume any splits are in the form discussed in Lemma 4. Consider the case where a split exists. We must show that this implies that given any sub-domain of $vars(C')$, if it is GAC with respect to C', any assignment to $\boldsymbol{V_r}$ can be extended to a valid assignment of $vars(C')$ with respect to both C' and the representation constraints of all the elements of R. If this was not the case, this would mean there was a sub-domain of $vars(C')$ which is GAC with respect to C' but there was an assignment to one of the $\boldsymbol{V_r}$ which could not be extended to a valid assignment of $vars(C')$. This would mean also it could not be extended to a valid assignment which satisfied all the constraints in a split of C'. As no variable is in more than one constraint of the split, this would mean that at least one constraint in the split had an assignment which was not satisfiable. However each constraint in the split contains at most one variable from $\boldsymbol{V_i}$, so that variable can be pruned, and therefore one constraint in the split of C' was not GAC, and therefore neither was C'. □

Corollary 1. *The occurrence representation is perfect when all variables in the constraints $A \cup B = C, A \cap B = C$ and $A \subseteq B$ are represented by the occurrence representation, and the (multi)set variables A,B and C are unsized.*

Proof. If A, B and C are represented by *occurrence* representation with vectors A',B' and C', then each of the constraints trivially splits to satisfy Theorem 5, for example for $A, B, C = MS(\{1, \ldots, n\}, 1, [0, n])$, $A \cup B = C$ becomes the set of constraints $\{A'[i] \wedge B'[i] = C'[i] | i \in \{1, \ldots, n\}\}$. □

Although we shall not prove it here, the explicit representation is not perfect for any of these constraints. This can be seen intuitively, as all these constraints involve examining every variable in representation to know if a certain element is in the set or not.

5 Conclusion

This paper has extended the study of representations by giving an implementation independent definition of representations and shown how this can be used to usefully compare representations both to each other in both a problem-dependent and problem-independent manner. Of particular interest is that a previously unstudied representations of sets and multisets has been proved to perform better than two frequently used representations. Further, we have shown that for problems which involve most common set constraints excluding cardinality, the occurrence representation performs as well as the theoretically best representation. The work in this paper is being further expanded to cover other families of constraints, in particular quantified constraints, and also other types of variables, such as graphs, are being further investigated.

References

1. C. Bessiere, E. Hebrard, B. Hnich, and T. Walsh. Disjoint, partition and intersection constraints for set and multiset variable. In *Proceedings of CP04*, 2004.
2. C. Bessière and J.-C. Régin. Arc consistency for general constraint networks: Preliminary results. In *IJCAI (1)*, pages 398–404, 1997.
3. A. M. Frisch, C. Jefferson, B. Martínez Hernández, and I. Miguel. The rules of constraint modelling. In *Proceedings of the 19th International Joint Conferences on Artifical Intelligence*, 2005.
4. C. Gervet. Conjunto: constraint logic programming with finite set domains. In M. Bruynooghe, editor, *Logic Programming - Proceedings of the 1994 International Symposium*, pages 339–358, Massachusetts Institute of Technology, 1994. The MIT Press.
5. W. Harvey and P. Stuckey. Improving linear constraint propagation by changing constraint representation. *Constraints*, 8[2]:173–207, 2003.
6. B. Hnich. *Function Variables for Constraint Programming.* PhD thesis, Uppsala, 2004.
7. B. Hnich, B. Smith, and T. Walsh. Dual modelling of permutation and injection problems. *Journal of Artificial Intelligence Research, Volume 21.*
8. T. Walsh. Consistency and propagation with multiset constraints: A formal viewpoint. In *Proceedings of CP*, 2003.

Modelling and Solving Temporal Reasoning as Propositional Satisfiability *

Duc Nghia Pham, John Thornton, and Abdul Sattar

Institute for Integrated and Intelligent Systems
Griffith University, Queensland, Australia
{d.n.pham, j.thornton, a.sattar}@griffith.edu.au

Abstract. Recent research has shown that it is often preferable to encode real-world problems as propositional satisfiability (SAT) problems, and then solve using general purpose solvers. In this way the efficiencies of SAT technology can be exploited, and the development of specialised solution techniques can be avoided. However, in the interval algebra (IA) domain of temporal reasoning, the state-of-the-art still involves the use of specialised techniques that exploit the particular structure of interval relations.
In this paper we investigate the feasibility of representing and solving IA problems as SAT problems. We firstly introduce two methods of representing IA as a constraint satisfaction problem (CSP), and then use three SAT-encoding schemes to produce six different IA to SAT representations. In an empirical study, we examine the performance of existing SAT local and complete search solvers on these SAT representations, and perform a comparison with solvers that operate on native IA representations. Our results show that the best performance over a range of algorithms is produced using a support SAT encoding of a point algebra-based CSP. The results also show that a state-of-the-art complete SAT solver (zChaff) can solve these instances significantly faster than existing IA solvers working on equivalent native IA representations.

1 Introduction

Representation and reasoning with time information, or *temporal reasoning*, is a fundamental research area in computer science and artificial intelligence. Basic tasks in this domain include the design and development of efficient reasoning methods for determining consistency of temporal representations, and effectively answering temporal queries. More generally, results from temporal reasoning research have been successfully applied in many real world AI applications such as planning, plan recognition, natural language understanding, and medical diagnosis [15].

In this paper we are specifically concerned with the interval algebra (IA) representation of the temporal reasoning problem [1]. This is firstly because IA offers considerable expressivity in terms of representing qualitative information and secondly because it is the most popular and well studied temporal reasoning formalism. Existing IA temporal reasoning techniques are generally based on the backtracking approach (proposed

* We would like to thank Peter van Beek and Jochen Renz for helpful comments on the earlier version of this paper.

by Ladkin and Reinefeld [10]), which uses path consistency as forward checking. Although this approach has been further improved [15, 22], it and its variants still rely on path consistency checking at each step to prune the search space. This *native* IA approach has the advantage of being fairly compact, but is disadvantaged by the overhead of continually ensuring path-consistency. Additionally, the native IA representation of variables and constraints means that state-of-the-art local search and complete search heuristics (such as unit propagation look ahead in Satz [12] or no-good recording and non-chronological backtracking in Chaff [14]) cannot be easily transferred to the temporal domain.

In practice, existing native IA backtracking approaches are only able to find consistent solutions for relatively small general IA instances [21, 19]. This has motivated research in applying stochastic local search techniques (SLS) to the IA problem, to see if performance increases over complete search observed in other SAT and CSP domains can be translated to IA. The first step in this direction was taken by Thornton *et al.* [19], with the development of the *end-point ordering* model, specifically designed to represent IA problems in a form suitable for processing by SLS. In this research the TSAT local search algorithm was shown to significantly outperform an existing complete search technique on a set of larger, more difficult IA problems. However, the end-point ordering model, like the native IA model, has a specialised structure that is carefully exploited by the TSAT algorithm. This means it is not a suitable representation for the application of general purpose techniques.

In this paper we ask the question whether the representation of IA problems using specialised models that require specialised algorithms is necessary in the general case. Given the development of such approaches takes considerable effort, we would expect significant performance benefits to result. To answer this question, we look at expressing IA as a propositional satisfiability problem. This enables us to apply a range of state-of-art SAT solvers and to compare the performance of these with the existing native IA approaches. According to our understanding, it appears that no explicit and thorough work has tried to formulate temporal problems as SAT instances. Nebel and Bürckert [16] pointed out that qualitative temporal instances can be translated to SAT instances but that such a translation causes an exponential blowup in problem size. Hence, no further investigation was provided in their work.[1]

A second issue addressed by the paper is the discovery of the best SAT representation for IA. This question divides into two parts: firstly the development of an appropriate CSP representation of IA (i.e. one suitable for the efficient application of general purpose solution techniques), and secondly the best choice of a CSP to SAT encoding. One of the main contributions of the paper is the development of a *point-based* CSP encoding of the IA problem which uses point algebra-based relations [23] but retains the full expressivity of IA. This is compared to a more straightforward interval-based model. We also extend the literature on the relative performance of existing CSP to SAT encodings by giving an empirical comparison of three encodings for each of our CSP models and for both complete and local search approaches.

[1] Recent independent work [6] has proposed representing IA as SAT, but the authors do not specify the transformation in detail, and fail to provide an adequate empirical evaluation.

The remainder of the paper is structured as follows: in the next section we review the basic definitions of IA, and in Section 3 we introduce the two methods to transform IA instances to CSP instances. Using these two transformation methods, combined with three CSP to SAT encodings, six IA to SAT encodings are presented. In Section 4.1 we describe the generation of our test set instances and in Sections 4-4.5 we present an empirical study to evaluate the performance of these SAT encodings relative to each other, and the performance of existing complete and SLS SAT solvers in comparison to the native IA backtracking and TSAT solvers. Finally, Section 5 presents the conclusions and the future research suggested by this study.

2 Interval Algebra

Interval Algebra [1] is the most commonly used formalism to represent temporal interval events. It consists of a set of 13 atomic relations between two time intervals: $\mathcal{I} = \{eq, b, bi, m, mi, o, oi, d, di, s, si, f, fi\}$ (see Table 1). Indefinite information between two time intervals can be expressed as a subset of $\mathcal{I}$ (e.g. a disjunction of atomic relations). For example, the statement *"Event X can happen either before or after event Y"* can be expressed as $X\{b, bi\}Y$. Hence there are a total of $2^{|\mathcal{I}|}$ = 8,192 possible relations between pairs of temporal intervals.

Atomic relation	Symbol	Diagram of meaning	PA representation
X before *Y*	b	X Y	$X^- < Y^-, X^- < Y^+$
Y after *X*	bi		$X^+ < Y^-, X^+ < Y^+$
X meets *Y*	m	X Y	$X^- < Y^-, X^- < Y^+$
Y met by *X*	mi		$X^+ = Y^-, X^+ < Y^+$
X overlaps *Y*	o	X	$X^- < Y^-, X^- < Y^+$
Y overlapped by *X*	oi	Y	$X^+ > Y^-, X^+ < Y^+$
X during *Y*	d	X	$X^- > Y^-, X^- < Y^+$
Y includes *X*	di	Y	$X^+ > Y^-, X^+ < Y^+$
X starts *Y*	s	X	$X^- = Y^-, X^- < Y^+$
Y started by *X*	si	Y	$X^+ > Y^-, X^+ < Y^+$
X finishes *Y*	f	X	$X^- > Y^-, X^- < Y^+$
Y finished by *X*	fi	Y	$X^+ > Y^-, X^+ = Y^+$
X equals *Y*	eq	X	$X^- = Y^-, X^- < Y^+$
		Y	$X^+ > Y^-, X^+ = Y^+$

Table 1. The thirteen IA atomic relations

The four operators of IA: *union* (denoted by $\cup$), *intersection* (denoted by $\cap$), *inversion* (denoted by $^{-1}$), and *composition* (denoted by $\circ$), can be defined as follows:

$$\forall\, X, Y : X(R_1 \cup R_2)Y \leftrightarrow (XR_1Y \vee XR_2Y)$$

$$\forall X, Y : X(R_1 \cap R_2)Y \leftrightarrow (XR_1Y \wedge XR_2Y)$$

$$\forall X, Y : X(R_1^{-1})Y \leftrightarrow YR_1X$$

$$\forall X, Y : X(R_1 \circ R_2)Y \leftrightarrow \exists Z : (XR_1Z \wedge ZR_2Y).$$

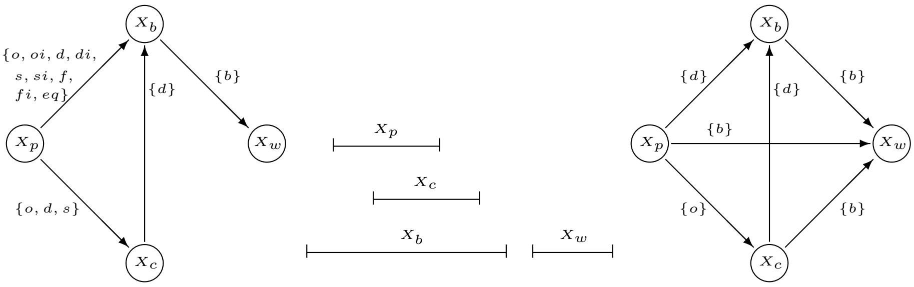
Hence, the *intersection* and *union* of any two temporal relations (R_1, R_2) are simply the standard set-theoretic intersection and union of the two sets of atomic relations describing R_1 and R_2, respectively. The *inversion* of a temporal relation R is the union of the inversion of each atomic relation $r_i \in R$. The *composition* of any pair of temporal relations (R_1, R_2) is the union of all results of the composition operation on each pair of atomic relations (r_{1i}, r_{2j}), where $r_{1i} \in R_1$ and $r_{2j} \in R_2$. The full composition results of these IA atomic relations are available in [1].

An IA network can be modelled as a temporal CSP (TCSP), where each interval event is a CSP variable with a domain of ordered pairs of real numbers and each binary constraint C_{ij} is labelled with the interval relations between the i^{th} and j^{th} intervals [15]. In general, the solution for a standard CSP is an assignment of domain values to all variables such that all the constraints are satisfied. Unfortunately, as the domain of an IA interval is infinite, this representation is not appropriate for a discrete domain CSP or SAT solver. However, the problem can be relaxed such that an $\mathcal{I}$-instantiation of a given IA network is an assignment of interval relations to all binary constraints in the corresponding TCSP. An $\mathcal{I}$-instantiation is *singleton*, also known as a *scenario*, iff each binary constraint is assigned with exactly a single atomic relation. An IA network with n interval variables is *globally consistent* iff it is strongly n-consistent [13]. Hence, the ISAT problem of determining whether a given IA network is satisfiable, becomes the problem of determining whether a *globally consistent* $\mathcal{I}$-instantiation of the corresponding TCSP exists [1, 15]. ISAT is the *fundamental* reasoning task in the temporal reasoning community because all other interesting reasoning problems can be reduced to it in polynomial time [7] and it is one of the most important tasks in practical applications [22].

Figure 1(a) shows an example of an IA network expressing the situation: *"Fred was reading the paper while eating his breakfast. He put the paper down and drank the last of his coffee. After breakfast he went for a walk."*.[2] In this example, variables X_p, X_b, X_c and X_w represent the interval that Fred is *reading the paper*, *eating the breakfast*, *drinking coffee* and *walking* respectively. Figure 1(b) shows a consistent solution of this network on the timeline and figure 1(c) shows the corresponding $\mathcal{I}$-instantiation of that solution.

As ISAT is known to be NP-complete [23], the application of some sort of *exhaustive search method* is generally required to determine the satisfiability of a full IA network. Ladkin and Reinefeld [10] proposed an efficient backtracking approach to solve the ISAT problem by enforcing path consistency as forward checking at every branching node. This allows the elimination of relations that are path inconsistent with the current partial solution. They also pointed out that the instantiation of each constraint can be extended from atomic relations to any set of relations for which path consistency guarantees global consistency, and hence considerably reduced the branching factor of the

[2] This example was originally used in [21].



(a) IA network (b) timeline solution (c) consistent scenario

Fig. 1. An example of an IA network and its consistent solution.

algorithm. In addition, various variable and value ordering techniques were developed and empirically shown to significantly improve overall performance [21, 11, 15].

Recently, Thornton *et al.* [19] developed a new transformation method, called *end-point ordering*, that reformulates IA networks into CSPs. In this model variables are interval end-points and constraints are end-point relations as defined in Table 1. The domain of each end-point variable is defined as the integer value position or rank of that variable within the *total ordering of all end-points* [19]. To solve these problems, a specialised TSAT local search algorithm was developed that exploits the structure of the end-point domains and constraints. The main difficulty with this approach is the generation of very large variable domains (representing all possible orderings of each interval). Without the special TSAT pruning heuristics a standard general purpose solver would not prove competitive. Therefore, neither native IA or the end-point ordering models are appropriate for use with a general purpose SAT or CSP solver. For this reason we decided to explore the development of alternative representations that could produce variables with reasonable domain sizes and easily represented constraints.

3 Encoding IA into SAT

Recent research has shown that modelling and solving hard combinatorial problems as SAT instances can produce significant performance benefits over solving problems in their original form [9, 8, 18]. This at least indicates that encoding and solving IA problems as SAT instances using state-of-the-art SAT solvers is a promising line of enquiry.

A common approach to encode combinatorial problems into SAT is to divide the task into two steps: (i) modelling the original problem as a CSP and (ii) mapping the new CSP into SAT. In the next two subsections, we propose two transformation methods to model IA networks as CSPs such that these CSPs can be feasibly translated into SAT. We then discuss three SAT encoding schemes to map the CSP formulations of IA networks into SAT. This results in six different approaches to encode IA networks into SAT.[3]

[3] The proof for these reformulating methods will appear in a longer version of this paper.

3.1 The Interval-Based Transformation Method

A straightforward method to formulate IA networks as CSPs is to represent each arc between a pair of intervals in the original IA network as a CSP variable. We then limit the domain values of each CSP variable to the set of permissible IA atomic relations for that arc, rather than the set of all subsets of $\mathcal{I}$ used in existing IA approaches. This allows us to reduce the domain size of each CSP variable from 2^{13} to a maximum of 13 values. In addition, an instantiation of the new model is now a singleton $\mathcal{I}$-instantiation, as a property of a CSP is that one and only one value can be assigned to a variable at any given time. Hence, the global constraint that an $\mathcal{I}$-instantiation has to be *globally consistent* becomes equivalent to it being *path consistent*, as a singleton $\mathcal{I}$-instantiation is globally consistent iff it is path consistent [21].

In his original work, Allen [1] proposed a path consistency algorithm for a regular $\mathcal{I}$-instantiation that repeatedly computes

$$R_{ik} = R_{ik} \cap (R_{ij} \circ R_{jk})$$

for all triples of intervals (i, j, k) until no further change occurs or until $R_{ik} = \emptyset$. These operations remove all the relations that cause an inconsistency between any triple (i, j, k) of intervals. If $R_{ik} = \emptyset$, then the original $\mathcal{I}$-instantiation is path inconsistent.

For a singleton $\mathcal{I}$-instantiation, the algorithm can be simplified, without loss of completeness, so that we only need to check

$$R_{ik} \subset (R_{ij} \circ R_{jk})$$

for all triples of interval $(i < j < k)$ once. The intersection ($\cap$) operation is unnecessary as R_{ik} is instantiated with exactly one atomic relation.

Formally, using this *interval*-based reduction method, the corresponding CSP of a given IA network is defined as follows:

Definition 1. *Given an IA network with n intervals, the corresponding interval-based CSP is (X, D, C), where $X = \{\upsilon_{ij} \mid i, j \in [1..n], i < j\}$; each variable υ_{ij} represents a relation between two intervals i and j, having a domain $D_{ij} = R_{ij}$; and C consists of the following constraints:*

$$\upsilon_{ij} = x \wedge \upsilon_{jk} = y \Longrightarrow \upsilon_{ik} \in \{z_1, ..., z_m\}, \quad (i, j, k \in [1..n], i < j < k) \qquad (1)$$

where $\{z_1, ..., z_m\} = D_{ik} \cap (x \circ y)$. Note that R_{ij} is the IA relation between i and j; and $x, y \in R_{ij}$.

This complete interval-based reduction method requires $O(n^3)$ time, where n is the number of intervals.

3.2 The Point-Based Transformation Method

Vilain and Kautz [23] proposed the Point Algebra (PA) to model qualitative information between time points. PA consists of a set of 3 atomic relations $\mathcal{P} = \{<, =, >\}$ and four operators defined in the similar manner to IA. As an interval event X can be represented as an ordered pair of end-points (X^-, X^+) where $X^- < X^+$, it follows that the relations between two intervals can be expressed as the relations between their end-points.

Although each atomic IA relation can be uniquely represented by a combination of relations on these points (see Table 1), representing non-atomic IA relations is more complex, as not all IA relations can be translated into point relations. For example, the following combination of point relations, $(X^- \neq Y^-) \wedge (X^- < Y^+) \wedge (X^+ \neq Y^-) \wedge (X^+ < Y^+)$, represents not only $X\{b,d\}Y$ but also $X\{b,d,o\}Y$. Therefore, PA can only cover 2% of IA [15].

However, we can simply introduce new constraints to prevent the representation of undesired IA relations in the CSP model. Such constraints are formed using the negation of the PA representation of the undesired atomic IA relation.[4] For instance, to disallow $X\{o\}Y$ in the above example, we would include the constraint:

$$\neg(X^- < Y^-) \vee \neg(X^- < Y^+) \vee \neg(X^+ > Y^-) \vee \neg(X^+ < Y^+)$$

Formally, using this *point*-based reduction method, the corresponding CSP of a given IA network is defined as follows:

Definition 2. *Let μ^r_{st} be the PA representation for an atomic IA relation r between two intervals s and t. Given an IA network with n intervals, the corresponding point-based CSP is (X, D, C), where $X = \{\upsilon_{ij} \mid i, j \in [1..2n], i < j\}$; each variable υ_{ij} represents a relation between two points i and j, having a domain $D_{ij} \in \mathcal{P}$; and C consists of the following constraints:*

$$\upsilon_{ij} = x \wedge \upsilon_{jk} = y \Longrightarrow \upsilon_{ik} \in \{z_1, ..., z_m\}, \quad (i, j, k \in [1..n], i < j < k) \tag{2}$$

$$\neg\mu^r_{st}, \quad (r \notin R_{st}) \tag{3}$$

where $\{z_1, ..., z_m\} = D_{ik} \cap (x \circ y)$. Note that x, y, z are PA atomic relations; s, t are intervals in the original problem and R_{st} is the relation between them.

3.3 Mapping CSP Representations of IA into SAT

Using either of the above CSP formulations, an IA network can be easily encoded as a SAT instance, where each Boolean variable υ^r_{ij} represents an assignment of domain value r to a CSP variable υ_{ij}. Two sets of at-least-one (ALO) and at-most-one (AMO) clauses can then be used to ensure that each CSP variable can only be instantiated with exactly one value at any time. It is common practice to encode a general CSP into SAT without the AMO clauses, thereby allowing CSP variables to be instantiated with more than one value [24]. A CSP solution can then be extracted by taking any single SAT-assigned value for each CSP variable. However, our two CSP reduction methods depend on the fact that each CSP variable can only be instantiated with exactly one value at any time. This maintains the completeness of formulation by ensuring not only the correctness of our translation of path consistency constraints (required by intersection

[4] Table 1 shows the PA representations of 13 IA atomic relations.

($\cap$) operations) but also the global consistency of the solution. Hence, the AMO clauses cannot be removed from our translation.

A natural way to encode the path consistency constraints, i.e. constraints (1) and (2) above, is to add the following support (SUP) clause for each pair of domain values (x, y) for the two CSP variables $(\upsilon_{ij}, \upsilon_{jk})$:

$$\neg\upsilon_{ij}^{x} \vee \neg\upsilon_{jk}^{y} \vee \upsilon_{ik}^{z_1} \vee ... \vee \upsilon_{ik}^{z_m}$$

where $\{z_1, ..., z_m\} = D_{ik} \cap (x \circ y)$. Note that we use the IA composition table for the interval-based reduction method and the PA composition table for the point-based reduction method [23].

For the extra, but essential, constraints that forbid undesired IA relations in the point-based model, i.e constraints of type (3) that cannot be excluded in a standard PA representation, we add a forbidden (FOR) clause for each of undesired IA relations involving the CSP variable in the original problem. The FOR clause of an atomic relation r is defined based on the negation of the PA representation of that relation. For example, given the PA representation of $X\{o\}Y$ is

$$(X^- < Y^-) \wedge (X^- < Y^+) \wedge (X^+ > Y^-) \wedge (X^+ < Y^+)$$

then the forbidden clause to rule out the relation $X\{o\}Y$ is

$$\neg\upsilon_{X^-Y^-}^{<} \vee \neg\upsilon_{X^-Y^+}^{<} \vee \neg\upsilon_{X^+Y^-}^{>} \vee \neg\upsilon_{X^+Y^+}^{<}$$

We call the above the *support* encoding scheme as it encodes the support values of the original problem. In Gent's support encoding scheme [5], the support clauses are necessary for both implication directions of the CSP constraints. However, in our scheme, only one SUP clause is needed for each triple of intervals $(i < j < k)$, and not for *all* permutation orders of this triple.

Formally, the SAT support encoding scheme for IA networks is defined as follows:

Proposition 1. *The support SAT-encoded instance of a given interval-based representation of an IA network consists of appropriate ALO, AMO and SUP clauses. The support SAT-encoded instance of a point-based representation is defined in a similar manner with the use of extra FOR clauses.*

Another way of representing CSP constraints as SAT clauses is to encode the conflict values, e.g. nogoods, between any pair of CSP variables [8, 24]. This *direct* encoding scheme for IA networks can be derived from our support encoding scheme by replacing the SUP clauses with the conflict (CON) clauses. If we represent SUP clauses between a triple of intervals $(i < j < k)$ as a 3D array of allowable values for the CSP variable υ_{ik} given the values of υ_{ij} and υ_{jk}, then the CON clauses can be defined as:

$$\neg\upsilon_{ij}^{x} \vee \neg\upsilon_{jk}^{y} \vee \neg\upsilon_{ik}^{z_m}$$

where $z_m \in \{D_{ik} - \{x \circ y\}\}$. The *multivalued* encoding [18] is a variation of the direct-encoding, where all AMO clauses are omitted. As discussed earlier, we did not consider such an encoding because in the IA transformations the AMO clauses play a necessary role.

Proposition 2. *The direct SAT-encoded instance of a given IA network is derived from the support SAT-encoded instance of that network by replacing SUP clauses with the appropriate CON clauses.*

A more compact version of the direct encoding is the *log* encoding [8, 24]. Here, a Boolean variable x_{ij} is true iff the corresponding CSP variable X_i is assigned a value in which the j-th bit of that value is 1. We can linearly derive log encoded IA instances from direct encoded IA instances by replacing each Boolean variable in the direct encoding with its *bitwise representation*. As a single instantiation of the underlying CSP variable is enforced by the bitwise representation, the AMO and ALO clauses can be omitted. However, extra bitwise prevention (PRE) clauses are needed (if necessary) to prevent bitwise representations of undesired Boolean variables from being instantiated. For example, if the domain of variable X is 3 then we have to add the clause $\neg x_{30} \vee \neg x_{31}$ to prevent the fourth value from assigning to X.

Proposition 3. *The log SAT-encoded instance of a given IA network is derived from the direct SAT-encoded instance of that network by replacing each Boolean variable with its bitwise representation, removing all AMO and ALO clauses and adding PRE clauses as necessary.*

4 Experimental Study

4.1 Generating SAT-encoded Test Instances

As a large collection of IA benchmarks is not available, the majority of work in the area has relied on randomly generated IA problems [22, 15]. This reliance on randomly generated problems has the advantage of allowing the average difficulty of a problem set to be controlled. We therefore based our empirical study on random problems generated using Nebel's $A(n, d, s)$ model [15]. The resulting instances are temporal constraint graphs with n nodes (i.e. intervals) and an average degree of d constrained arcs (i.e. interval relations). Constrained arcs are then labelled with an average of s IA atomic relations, where $1 \leq s \leq 12$. Unconstrained arcs are labelled with all 13 IA atomic relations.

Unlike the $S(n, d, s)$ model used in earlier studies (e.g. [19]), the $A(n, d, s)$ model allows us to control the difficulty of generated problems. Nebel [15] suggested that the problem instances can be generated in the phase transition using the $A(n, d, s)$ model with the values of d and s fixed to 9.5 and 6.5, respectively. It is worthy to note that the phase transition of a problem type is the border between two regions: one is where most of the problems have many solutions and it is relatively easy to solve; and one is where most of the problems are unsatisfiable and it is also relatively easy to prove [3]. In addition, research has empirically showed that instances in the phase transition are harder to solve for both complete and incomplete search solvers. We therefore used these settings to limit the study to only consider harder instances.

To investigate the performance effects of our six different SAT-encoding schemes, we followed Nebel's original study and randomly generated three test sets of 20, 30 and 40 nodes, each containing 100 satisfiable instances. We then pre-processed these instances using the path consistency algorithm before encoding them into SAT. We found this pre-processing significantly reduced the problem size of SAT-encoded IA instances both in terms of the number of variables and clauses.

Table 2 shows the average problem size of these instances together with the average time required to encode them into SAT instances. These results indicate the point-based support encoding can produce the smallest SAT-encoded instances within the shortest time window.

Problem	Reduction	Encoding	#vars	#clauses	time (secs)
$n = 20$	Interval	support	$1,685$	$75,667$	0.06
$d = 9.5$	based	direct	$1,685$	$695,713$	0.19
$s = 6.5$		log	639	$688,180$	0.43
	Point	support	$1,954$	$42,157$	0.02
	based	direct	$1,954$	$81,094$	0.03
		log	$1,299$	$79,264$	0.03
$n = 30$	Interval	support	$4,484$	$371,486$	0.19
$d = 9.5$	based	direct	$4,484$	$3,697,416$	0.83
$s = 6.5$		log	$1,577$	$3,675,095$	2.37
	Point	support	$4,741$	$173,746$	0.06
	based	direct	$4,741$	$340,366$	0.10
		log	$3,146$	$335,800$	0.14
$n = 40$	Interval	support	$8,421$	$1,019,320$	0.45
$d = 9.5$	based	direct	$8,421$	$10,415,055$	2.26
$s = 6.5$		log	$2,877$	$10,351,622$	6.80
	Point	support	$8,637$	$443,354$	0.14
	based	direct	$8,637$	$873,715$	0.23
		log	$5,744$	$865,344$	0.36

Table 2. Interval versus Point Based Encodings: A comparison on problem generation.

4.2 SAT Solver Selection

Table 3 shows the results of zChaff [14], PAWS [20] and MV-PAWS [17] for the three test sets generated in Section 4.1.[5] We ran PAWS and MV-PAWS for $1,000$ runs on each 20 node instance and 100 runs on each 30 and 40 instance. All three methods were timed out after $1,200$ seconds for each run.

We chose PAWS as our SLS solver [20], as PAWS was recently shown to be one of the most competitive SAT solvers on a range of larger and more difficult problems, while also requiring considerably less effort in terms of parameter tuning than other

[5] All our experiments were performed on a Sun supercomputer with $8 \times$ Sun Fire V880 servers, each with $8 \times$ UltraSPARC-III 900MHz CPU and 8GB memory per node.

comparable weighting schemes. For complete search, we chose zChaff [14] version 2004.11.25 as it has won the championship in the SAT competition for three consecutive years. In addition, we included a recently developed version of PAWS, MV-PAWS [17], which implements the same local search heuristic as PAWS, but is specifically designed to exploit CSP structure in SAT-encoded instances. MV-PAWS does this by automatically recognising the structure of CSP variables in a SAT problem and ensuring that each underlying CSP variable is only ever instantiated with a single value during the search. This built-in MV-PAWS mechanism means it can discard all ALO and AMO clauses, as they are now redundant.

			zChaff	PAWS			MV-PAWS		
Problem	Reduction	Encoding	$\frac{mean}{time}$	Param	$\frac{mean}{time}$	$\frac{mean}{flips}$	Param	$\frac{mean}{time}$	$\frac{mean}{flips}$
$n = 20$	Interval	support	0.17	33	1.42	67,441	100	0.85	15,955
$d = 9.5$	based	direct	2.66	29	18.04	133,572	n/a	n/a	n/a
$s = 6.5$		log	1,200	67	65.82	190,436	n/a	n/a	n/a
	Point	support	0.07	52	1.13	124,388	100	0.26	11,696
	based	direct	0.17	50	3.53	246,886	n/a	n/a	n/a
		log	3.49	68	1.16	54,734	n/a	n/a	n/a
$n = 30$	Interval	support	2.17	40	42.83	533,668	100	20.59	134,887
$d = 9.5$	based	direct	84.54	31	147.94	382,124	n/a	n/a	n/a
$s = 6.5$		log	1,200	69	271.55	278,265	n/a	n/a	n/a
	Point	support	0.53	61	23.28	941,819	100	3.53	67,462
	based	direct	1.95	60	78.80	1,691,735	n/a	n/a	n/a
		log	104.90	70	28.57	440,037	n/a	n/a	n/a
$n = 40$	Interval	support	14.61	40	210.38	1,274,465	100	120.13	389,524
$d = 9.5$	based	direct	354.99	n/a	n/a	n/a	n/a	n/a	n/a
$s = 6.5$		log	1,200	n/a	n/a	n/a	n/a	n/a	n/a
	Point	support	3.58	80	179.14	2,821,202	100	34.36	287,986
	based	direct	13.74	n/a	n/a	n/a	n/a	n/a	n/a
		log	914.36	n/a	n/a	n/a	n/a	n/a	n/a

Table 3. A comparison of Interval versus Point Based Encodings.

4.3 Results for Different Encodings

Overall, the results in Table 3 and graphed in Figure 2 clearly show that the point-based transformation produced better results that the interval-based encoding, regardless of problem size, solver or the SAT encoding method employed. Similarly, the support encoding produced the best performance of all three SAT encoding schemes, regardless of problem size, solver[6] or transformation method (this is consistent with earlier work

[6] As the support encoding was so strongly favoured by PAWS and zChaff we did not test MV-PAWS on the other encodings.

of [5]). Putting this together leads to the strong conclusion that a combination of point-based transformation and support encoding produces the best results, at least for the randomly generated IA problems considered.

The superior performance of the support encoding can be partly explained by the significantly smaller number of clauses generated (on average about ten times less than for direct or log encoded instances). However, it should be noted that the search space (i.e. the number of variables) of support and direct encoded instances are the same, whereas the search space of log encoded instances is $O(n \times (|s| - log|s|))$ times smaller [8]. A further possible reason for the superiority of the support encoding (suggested by Gent [5]) is the reduced bias to falsify clauses, i.e. the numbers of positive and negative literals in support encoding are more balanced than in direct encoding and hence this may prevent the search from resetting variables to false shortly after they are set to true.

Although our experiments confirmed the superiority of direct over log encoding (as per Hoos [8]) for the interval-based transformation, our results differed for the point-based transformation, where log encoding was found to be better than direct encoding, both in terms of runtimes and flips. To investigate this further, we used two features of the search space originally developed in [8]'s study: the standard deviation of the objective function (*sdnclu*) and the local minima branching along SLS trajectories (*blmin*) (we did not look at the solution density as this was the same for both encodings). Intuitively, the larger the *sdnclu* and *blmin* values are, the more effective the SLS solver. However, this hypothesis did not fit with our results. This may be explained by the reduced size of the point-based log clauses, resulting from the smaller number of PA atomic relations (in comparison to IA).

4.4 Results for SAT Solvers

As with the encoding results, a comparison of the relative performance of the three solvers produces a fairly unambiguous picture: zChaff outperforms MV-PAWS, and MV-PAWS outperforms PAWS, with relative performance being fairly independent of the transformation or SAT encoding method employed. The superior performance of MV-PAWS over PAWS confirms the results of [17], where MV-PAWS had a similar advantage on a range of SAT-encoded non-temporal CSP instances. This also confirms the results of earlier studies showing that SAT algorithms which exploit CSP structure generally produce better performance (e.g. [4, 2]).

However, the overall domination of zChaff is more surprising, as the earlier TSAT study [19] indicated that local search has an advantage over complete search in the temporal domain. In these results, zChaff is not only better across the board, but appears to be scaling as well or better than both of the local search approaches. It should also be considered that there is a version of zChaff [2] (analogous to MV-PAWS) that also exploits underlying CSP structure, and we would expect this solver, when released, to produce even better performance on these SAT-encoded instances.

4.5 SAT versus Existing Approaches

The experimental results in the previous section have clearly shown a point-based transformation using a support SAT encoding is the better encoding, that MV-PAWS is the

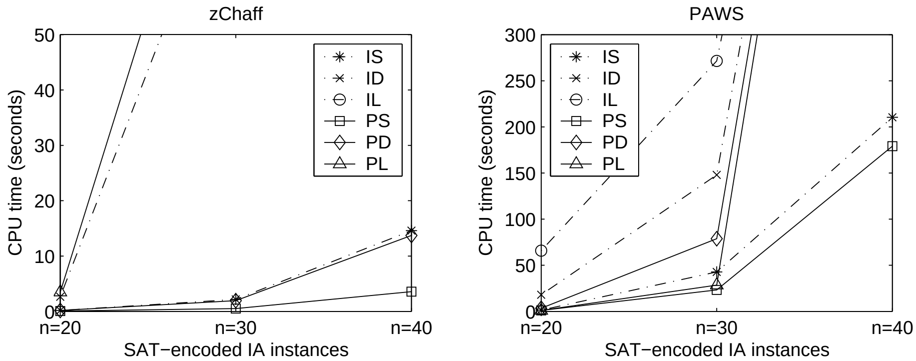


Fig. 2. Scalability of zChaff and PAWS on different SAT encodings.

better local search solver, and that zChaff is the better overall solver. The final question to address is how these SAT approaches compare to the existing state-of-the-art specialised IA solvers, namely TSAT and Nebel's backtracking/path consistency algorithm.

For this study we generated three test sets of 80, 90 and 100 nodes using the same method discussed in Section 4.1 (with each set containing 10 satisfiable instances). In addition to using zChaff and MV-PAWS on support encoded point-based transformations, we used TSAT on its end point ordering representation and used two variants of Nebel's backtracking algorithm (NBT+$\mathcal{I}$ and NBT+$\mathcal{H}$), on the native IA representations. NBT+$\mathcal{I}$ instantiates each arc with an atomic relation in $\mathcal{I}$, whereas NBT+$\mathcal{H}$ assigns a relation in the set $\mathcal{H}$ of *ORD-Horn* relations to each arc. Other heuristics used in Nebel's backtracking algorithm were set to default.

The results in Table 4 provide strong evidence that the approach of modelling IA problems as SAT has met with success. In particular, zChaff was the only method capable of solving all instances in the problem set within 1 hour. While Nebel's NBT+$\mathcal{H}$) was superior on the 80 node problems, it failed to scale up on the 90 and 100 node instances, and while TSAT remained competitive with zChaff in terms of median time, it proved unable to consistently solve all instances (falling to 76% success on the 100 node problems). Of secondary interest is that TSAT does appear to have the advantage over MV-PAWS, especially on the larger instances.

5 Conclusions and Future Work

In conclusion, the experiments indicate that our new point-based support encoding is the most suitable scheme to encode IA problems into SAT instances. The results further show that running a state-of-the-art complete SAT solver on such representations can produce superior results to two of the fastest specialised IA solvers reported in the current literature. This suggests that a SAT approach to solving larger and more difficult IA problems may be preferable to developing specialised representations and algorithms.

		Success	Time (secs)	
Problem	Solver	100%	$median$	$mean$
$n = 80$	zChaff	100	191.46	244.04
$d = 9.5$	MV-PAWS	90	416.41	925.41
$s = 6.5$	TSAT	87	27.01	594.66
	NBT+$\mathcal{H}$	100	64.81	210.06
	NBT+$\mathcal{I}$	50	$3,309.31$	$3,024.78$
$n = 90$	zChaff	100	393.77	576.67
$d = 9.5$	MV-PAWS	85	944.91	$1,364.66$
$s = 6.5$	TSAT	82	539.88	915.25
	NBT+$\mathcal{H}$	60	$1,792.86$	$2,238.46$
	NBT+$\mathcal{I}$	30	$3,600.00$	$2,581.89$
$n = 100$	zChaff	100	$1,132.23$	$1,120.03$
$d = 9.5$	MV-PAWS	64	$2,059.51$	$2,229.51$
$s = 6.5$	TSAT	76	$1,307.80$	$1,525.73$
	NBT+$\mathcal{H}$	30	$3,600.00$	$2,571.44$
	NBT+$\mathcal{I}$	20	$3,600.00$	$3,270.14$

Table 4. A comparison of SAT versus Existing Approaches.

In future work we anticipate that the performance of our SAT-based approach can be further improved by exploiting the special structure of IA problems in a manner analogous to the work on TSAT. The possibility also opens up of integrating our approach to temporal reasoning into other well known real world problem domains such as planning. Given the success of SAT solvers in many other real world domains, our work promises to expand the reach of temporal reasoning approaches for IA to encompass larger and more practical problems.

References

1. J. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26:832–843, November 1983.
2. Carlos Ansótegui, Jose Larrubia, and Felip Manyà. Boosting chaff's performance by incorporating CSP heuristics. In *Proceedings of the Ninth International Conference on Principles and Practice of Constraint Programming (CP-03)*, pages 96–107, 2003.
3. P. Cheeseman, B. Kanefsky, and W. Taylor. Where the really hard problems are. In *Proceedings of the Twelveth International Joint Conference on Artificial Intelligence (IJCAI-91)*, pages 331–337, 1991.
4. A. Frisch and T. Peugniez. Solving non-Boolean satisifiability problems with stochastic local search. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*, 2001.
5. I. Gent. Arc consistency in SAT. In *Proceedings of the Fifteenth European Conference on Artificial Intelligence (ECAI-02)*, pages 121–125, 2002.
6. K. Ghiathi and G. Ghassem-Sani. Using satisfiability in temporal planning. *WSEAS Transactions on Computers*, 3(4):963–969, 2004.

7. M. Golumbic and R. Shamir. Complexity and algorithms for reasoning about time: A graph-theoretic approach. *Journal of ACM*, pages 1108–1133, 1993.
8. H. Hoos. SAT-encodings, search space structure, and local search performance. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI-99)*, pages 296–302, 1999.
9. H. Kautz, D. McAllester, and B. Selman. Encoding plans in propositional logic. In *Proceedings of the Fifth International Conference on Principles of Knowledge Representation and Reasoning (KR-96)*, pages 374–384, 1996.
10. P. Ladkin and A. Reinefeld. Effective solution of qualitative interval constraint problems. *Artificial Intelligence*, 57(1):105–124, 1992.
11. Peter Ladkin and Alexander Reinefeld. Fast algebraic methods for interval constraint problems. *Annals of Mathematics and Artificial Intelligence*, 19:383–411, 1997.
12. C. Li and Anbulagan. Look-ahead versus look-back for satisfiability problems. In *Proceedings of the Third International Conference on Principles and Practice of Constraint Programming (CP-97)*, pages 341–355, 1997.
13. A. K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8:99–118, * 1977.
14. M. Moskewicz, C. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference (DAC-01)*, Las Vegas, 2001.
15. B. Nebel. Solving hard qualitative temporal reasoning problems: Evaluating the efficiency of using the ORD-Horn class. *Constraints*, 1(3):175–190, 1997.
16. B. Nebel and H. J. Bürckert. Reasoning about temporal relations: A maximal tractable subclass of Allen's Interval Algebra. *Journal of ACM*, 42(1):43–66, 1995.
17. D. N. Pham, J. Thornton, A. Sattar, and A. Ishtaiwi. SAT-based versus CSP-based constraint weighting for satisfiability. In *Proceedings of the Twentieth National Conference on Artificial Intelligence (AAAI-05)*, page to appear, 2005.
18. S. Prestwich. Local search on SAT-encoded colouring problems. *Lecture Notes in Computer Science*, 2003.
19. J. Thornton, M. Beaumont, A. Sattar, and M. Maher. A local search approach to modelling and solving Interval Algebra problems. *Journal of Logic and Computation*, 14(1):93–112, 2004.
20. John Thornton, Duc Nghia Pham, Stuart Bain, and Valnir Ferreira Jr. Additive versus multiplicative clause weighting for SAT. In *Proceedings of the Twentieth National Conference on Artificial Intelligence (AAAI-04)*, pages 191–196, 2004.
21. P. van Beek. Reasoning about Qualitative Temporal Information. *Artificial Intelligence*, 58:297–326, 1992.
22. P. van Beek and D. Manchak. The design and experimental analysis of algorithms for temporal reasoning. *Journal of Artificial Intelligence Research*, 4:1–18, 1996.
23. M. Vilain and H. Kautz. Constraint propagation algorithms for temporal reasoning. In *Proceedings of the Fifth National Conference on Artificial Intelligence (AAAI-86)*, pages 377–382, 1986.
24. Toby Walsh. SAT v CSP. In *Proceedings of the Sixth International Conference on Principles and Practice of Constraint Programming (CP-00)*, pages 441–456, 2000.